CS2106

AY25/26 Sem 1

by ngmh

Operating Systems

- OS is software that runs in kernel mode with direct access to hardware resources, other software runs in user mode
- User programs call OS using system call interface
- Kernel is a program with special features to deal with hardware and provide the syscall interface, with special code for interrupt handlers and device drivers
- Kernel code cannot use syscalls, normal libraries, or I/O
- Virtualisation: each program executes as if it has all resources to itself

OS Structures

- Monolithic: Kernel is one big special program, various services and components are integral part
- Good performance but complicated internal structure with highly coupled components
- Microkernel: Small and clean, only provides basic and essential facilities such as IPC, address space and thread management
- Higher level services are built on top, and run as server process outside of OS, using IPC to communicate
- More robust and extendible, with better isolation and protection, but lower performance
- Others include layered systems or client-server model

Virtual Machines

- Allows us to run multiples OSes on the same hardware at the same time, as well as debug / monitor them
- Virtual Machine: Software emulation / virtualisation of hardware, created and managed by hypervisor / Virtual Machine Monitor
- Type 1 Hypervisor: Provides individual virtual machines to guest OSes, bare metal hypervisor directly on hardware
- Type 2 Hypervisor: Runs in host OS, guest OS runs inside virtual machine

Process Abstraction

Memory Context: Function Call

- Stack Memory: New memory region to store information for a function invocation, described by a stack frame
- Stack Frame contains ra / PC, arguments / parameters, storage for local variables, other info
- Stack Pointer: Points to first unused location, usually in a specialised register
- Function Call Convention (Different ways to setup stack frame)
  - Prepare to make call: Caller passes parameters with registers or stack, then saves return PC on stack
  - Transfer control to callee: Callee saves old stack pointer, allocates space for local variables, and adjusts stack pointer to new stack top
  - On returning from function call: Callee places return result in register, and restores saved stack pointer
  - Transfer control back to caller using saved PC: Caller utilises return result, and continues execution
- Other information: Frame pointer, saved registers
- Frame Pointer: Facilitates access of various stack frame items, might be dedicated register, points to fixed location in a stack frame so that other items can be accessed as a displacement from it, each stack frame has one
- Saved registers: Use memory to hold GPR values when exhausted, so it can be used and restored, known as register spilling
- Function can spill registers it intends to use before function starts, to be restored at end of function
- Updated Steps
  - Prepare to make call: Caller passes parameters with registers or stack, then saves return PC on stack
  - Transfer control to callee: Callee saves registers used by callee, old stack pointer, old frame pointer, allocates space for local variables on stack, and adjusts stack pointer to new stack top, adjusts frame pointer
  - On returning from function call: Callee restores saved registers, FP, SP, places return result in register
  - Transfer control back to caller using saved PC: Caller utilises return result, and continues execution
- Stack Frame Illustration:
  - Free Memory; Local Variables (Stack pointer points above here)
  - Parameters; Saved Registers
  - Saved SP (Top of previous stack frame), (Frame pointer points above here)
  - Saved FP (Base address of previous stack frame)
  - Return PC
- Text memory: For instructions
- Data memory: For global / static variables

Process Management

- Process ID: Used to distinguish processes from each other, unique
- Process State: Processes can be either running, or not running, but can also be in a ready state
- Process Model: Set of states and transitions
- 5-Stage Model: New, Ready, Running, Blocked, Terminated
- Queuing Model: OS has ready and blocked queue with processes inside, awaiting execution
- Process Control Block (PCB) or Process Table Entry: Entire execution context for a process
- Kernel maintains PCB for all processes in a table
- Issues: Scalability and Efficiency

System Calls

- General System Call Mechanism
  - User program invokes library call using normal function call mechanism
  - Library call (usually in assembly) places system call number in designated location such as register
  - Library call executes special instruction (TRAP) to switch from user to kernel mode, saves CPU state
  - Appropriate system call handler is determined, using system call number as index, handled by dispatcher
  - System call handler is executed
  - System call handler ends, restores CPU state, returns to library call, switch back to user mode
  - Library call returns to user program via normal function return mechanism

Exception and Interrupt

- Exceptions can be caused by executing machine level instructions, e.g. Arithmetic / Memory Access errors
- Exceptions are synchronous, occurring due to execution
- Exception handler is executed, similar to a forced function call
- Interrupts are external events that can interrupt the execution of a program, usually hardware related
- Interrupts are asynchronous, independent of program execution
- Program execution is suspended, and an interrupt handler must be executed
- Control transfer to handler routine automatically, before being returned to

Process Abstraction in Unix

- Processes are identified by integer Process IDs
- Information: Process State (Running, Sleeping, Stopped, Zombie), Parent PID, Cumulative CPU time
- Process creation: Using fork(), duplicates executable image, copy on write
- Child only differs in PID, PPID, and fork() return value (PID of new process for parent, 0 for child)
- Both continue execution after fork()
- For a different program, can use exec() variations instead
- Master process: init, created in kernel at boot up, has PID 1, forms process tree
- Use exit() to end execution of process, 0 is normal exit
- On finish, most system resources used are released such as file descriptors, while basic process resources are not like PID and status as well as process accounting info since process table entry might still be needed
- Parent Child Synchronisation: Parent process waits for **immediate child process** to terminate using `wait(int *status)`
- Wait is blocking until at least one child terminates, and cleans up remainder of child system resources, has other variations as well
- On process exit, zombie is created since not all process info is deleted, wait() cleans it up
- Orphan: Parent terminates before child process: init becomes psuedo parent, then child termination sends signal to init which cleans it up with wait()
- Zombie: Child process terminates before parent, who did not wait(): Child becomes a zombie, and can fill up process table

Implementation of fork()

- Makes an almost exact copy of parent process
  - Create address space of child process
  - Allocate new PID
  - Create kernel process data structures like entry in process table
  - Copy kernel environment of parent process such as priority
  - Initialise child process context like PID, PPID, CPU time
  - Copy memory regions from parent such as Program, Data, Stack, but this is expensive
  - Acquire shared resources like open files, current working directory
  - Initialise hardware context for child processes by copying registers from parent process or others
  - Add child process to scheduler queue to wait to be run
- Copy on Write: Only duplicate a memory location when written to
- Memory is stored in memory pages and managed on a page level instead

Process Scheduling

Concurrent Execution

- Concurrent Processes: Multiple processes progress in execution at the same time, could be virtual or physical
- Timeslicing: Using 1 CPU, interleave instructions from both processes
- Multitasking needs to change context between processes, which incurs overhead
- Multiprocessor: Timeslicing on multiple CPUs at the same time
- Scheduling: How to organise processes to run
- Scheduler: Part of OS which makes scheduling decision
- Scheduling Algorithm: Algorithm used by scheduler
- Process Behaviour: Each process requires different amounts of CPU time for CPU-Activity and IO-Activity, can be either CPU/IO-Bound
- Processing Environment: Batch Processing (No interaction required, no need to be responsive), Interactive (User actively interacts with system, needs to be

responsive and consistent in responses), Real Time Processing (Have deadline to meet, usually periodic)
- Scheduling Algorithms: Influenced by processing environment, common criteria are Fairness (Each get fair share of CPU time, no starvation), Utilisation (All parts of computing system should be utilised)
- Scheduling Policies: Non-Preemptive / Cooperative (Process stays running until it blocks or gives up CPU), Preemptive (Process given fixed time quota to run, and gets suspended after quota is up)

Scheduling for Batch Processing

- No user interaction, non-preemptive scheduling is predominant
- Criteria: Turnaround time (Total time taken, considers waiting time), Throughput (Rate of task completion), CPU Utilisation (Percentage of time CPU spends working)
- First Come First Served (FCFS): Queueing system, no starvation, having longer tasks later can reduce waiting time
- Shortest Job First (SJF): Select task with total CPU time, use exponential average to predict, could cause starvation
- Shortest Remaining Time (SRT): Similar, uses remaining time instead

Scheduling for Interactive Systems

- Criteria: Response Time, Predictability (Variation in response time)
- Preemptive scheduling algorithms are used to ensure good response time, and scheduler needs to run periodically
- Timer interrupt handler is used to ensure scheduler can run periodically without being stopped by other programs
- Interval of Timer Interrupt (ITI): Typically a few ms, OS scheduler invoked on every timer interrupt
- Time Quantum: Execution duration given to a process, multiple of ITI, large range of values, can be variable
- Round Robin: Preemptive version of FCFS, time quantum is needed with timer interrupt
- Priority Scheduling: Run processes with higher priority first, might cause starvation, reducee priority of running processes to solve
- Multi Level Feedback Queue (MLFQ)
  - Adaptive, learns process behaviour, minimises response and turnaround time
  - Rules
    - If $p_a > p_b$, a runs
    - If $p_a = p_b$, a and b run in RR
    - New job has highest priority
    - If a job fully utilises its time slice, its priority is reduced
    - If a job gives up or blocks, its priority is retained
- Lottery Scheduling: Lotter for processing time, can choose how to distribute

Cases from Tutorial

- Lengthy CPU phase followed by IO intensive phase: Priority can sink during CPU intensive phase, so it might not get enough in the next phase; Solve by moving all process in the system to the highest priority periodically
- Process repeatedly gives up CPU just before time quantum, retains priority indefinitely; Solve by accumulating CPU usage across time quanta

Inter-Process Communication

- Allow cooperating processes to share information
- Shared Memory
  - First process creates shared memory region
  - Second process attaches this to its own memory space
  - Processes can now communicate using this memory region
  - OS is only involved in creation and attaching
  - Efficient and provides ease of use
  - Synchronisation of access is needed and implementation might be harder
  - For POSIX SHM in *nix, need to destroy after use; only if not attached to any process

```
int shmid = shmget(IPC_PRIVATE, 40, IPC_CREAT | 0600);
int *shm = (int*) shmat(shmid, NULL, 0);
shmdt((char*) shm);
shmctl(shmid, IPC_RMID, 0);
```

- Message Passing: One process prepares a message and sends it to another who receives it
- Message stored in kernel memory space, needs OS and syscalls
- Naming schemes determine how message gets to recipient
- Unix Pipes: Link input and output to one another, shared between processes, circular bounded byte buffer with synchronisation

```
int pipeFd[2];
pipe(pipeFd);
write(pipeFd[WRITE_END], str, strlen(str)+1);
int len = read(pipeFd[READ_END], buf, sizeof(buf));
close(pipeFd[READ_END]);
// close end not being used
```

- Unix Signal
  - Asynchronous notification regarding an event sent to a process or thread
  - Recipient must handle signal using a default set of handlers or user supplied handler
  - UNIX examples include Kill, Interrupt, Memory Error etc.
  - `signal(SIGSEGV, handler)`, `void handler(int signo)`

Threads

- Threads share memory context (text, data, heap) and OS context (PID and other resources)
- Identified by thread id, and have unique registers and stack
- Thread switching is more lightweight as only register and stack needs to be swapped (hardware context) v/s process context switch (OS, hardware, memory context)

Thread Models

- User Thread: Implemented as a user library, runtime system in process will handle thread related operations, kernel is not aware of threads in process
- Can have multithreaded program on any OS, generally more configurable and flexible
- Since scheduling is performed at process level, whole process gets blocked at once, cannot exploit multiple CPUs
- Kernel Thread: Implemented in OS, handled as system calls, and thread-level scheduling is possible
- More than 1 thread in the same process can run at the same time on multiple CPUs
- Thread operations are slower and more resource intensive as system calls
- Generally less flexible, and might be expensive or overkill
- Hybrid Thread Model: Have both, OS schedules kernel threads only, user thread can bind to kernel thread, offers greater flexibility
- Threads started as a software mechanism, but have hardware support on modern processors in the form of multiple sets of registers allowing them to run in parallel on the same core, known as simultaneous multi-threading (SMT)

POSIX Threads

- Standard defined by IEEE, but implementation is not specified
- pthread library, with methods like create, exit, join

```
int pthread_create(
    pthread_t* tidCreated,
    const pthread_attr_t* threadAttributes,
    void* (*startRoutine) (void*),
    void* argForStartRoutine);
int pthread_exit(void* exitValue);
int pthread_join(pthread_t threadID, void **status);
```

Synchronisation

- Designate code segment with race condition as critical section where only one process can execute in there at a time
- Properties
  - Mutual Exclusion: If a process is executing in the critical section, all other process are prevented from entering
  - Progress: If no process is in a critical section, one of the waiting process should be granted access
  - Bounded Wait: After a process requests to enter critical section, there exists an upper bound of number of times other processors can enter before it
  - Independence: Process not executing in critical section should never block other processes
- Incorrect Synchronisation can cause
  - Deadlock: All processes are blocked and no progress is made
  - Livelock: Processes keep changing state to avoid deadlock and make no other progress but are not blocked
  - Starvation: Some processes never get to make progress because they are perpetually denied necessary resources
- Can have assembly / HLL implementations or high level abstraction
- High Level Abstraction
  - Semaphore: General synchronisation mechanism; only behaviour is specified rather than implementation
  - Provides a way to block / sleep processes or unblock / wake them up
  - Semaphore contains an integer value and has two atomic semaphore operations, "counts how many resources are available"
  - Wait() blocks if not positive, and decrements
  - Signal() increments, and wakes up sleeping processes if any, never blocks
  - Invariant: $S_{current} = S_{initial} + signal(S) - wait(S)$ where it refers to number of signal() executed and wait() completed
  - Can have general or binary semaphores, with general semaphores used for convenience
  - Example: Wait before CS, Signal after completion, known as mutex
  - Deadlock is still possible with incorrect use of semaphores e.g. multiple overlapping semaphores
  - Common Alternative: Conditional variable, allow task to wait for certain event to happen, has ability to broadcast (wake up all waiting tasks), related to monitor

Synchronisation Implementations

- POSIX Semaphore: wait() and signal() implemented
- pthread has mutex as a binary semaphore with lock and unlock
- Also has conditional variables with wait, signal, and broadcast
- Most HLL have some level of thread and synchronisation support

Memory Abstraction

- Physical Memory Storage: Random Access Memory, treated as array of bytes, with unique index / physical address
- Executable contains code for text region, data layout for data region
- Transient Data: Valid only for a limited duration such as during function call e.g. parameters and local variables

- Persistent Data: Valid for duration of program unless explicitly removed e.g. global or constant variables and dynamically allocated memory
- Both types of data sections can grow or shrink during execution
- OS handles memory related tasks

Memory Abstraction

- Without abstraction, memory access is straightforward, but if two processes occupy the same memory, they will conflict
- Address Relocation: Recalculate memory references when process is loaded into memory, such as by adding offset, however loading time is slow and identifying memory references from integer constants is hard
- Base + Limit Registers: Use special base register as base of all memory references, memory references are computed as an offset for this, at loading time, base register is initialised to starting address of process memory space
- Limit register is used to indicate range of memory space of current process, accesses are checked against this limit
- However, every access incurs addition and comparison
- Logical Address: Avoid embedding physical memory address by using logical address as a view of how process views its memory space

Contiguous Memory Management

- Process must be in memory during execution, occupying a contiguous memory region, physical memory is large enough to contain entire process and memory space
- Support multitasking, and free up physical memory when full by removing terminated processes and swapping blocked processes to secondary storage
- Memory Partition: Contiguous memory region allocated to a single process
- Fixed-Size Partition: Physical memory split into fixed number of partitions to be occupied by processes
- Could cause internal fragmentation, and partition size needs to be large enough to contain an entire process
- Easy to manage and fast to allocate
- Variable-Size Partition: Partition is created based on actual size of process, OS keeps track of occupied and free memory regions
- Free memory space is known as hole, can cause external fragmentation with process creation or termination
- Flexible and removes internal fragmentation, but needs more information to be maintained and takes more time to allocate an appropriate region
- Allocation Algorithms: Search for hole with size large enough, and split it into new partition and a new hole
- First-Fit: First hole that is large enough, Best-Fit: Smallest hole that is large enough, Worst-Fit: Find the largest hole, Next-Fit: First-Fit but start from previously allocated partition
- Merge adjacent holes if possible, and moved occupied partitions around to consolidate them, should not be invoked too frequently
- e.g. OS uses linked list to maintain partition information (status, start address, length)
- Bitmap: Each unit of memory is marked free or allocated, no need to merge but overhead from bit manipulation

Buddy System

- Provides efficient splitting, locating, and de-allocation and coalescing
- Split block repeatedly in half to meet request to form sibling / buddy blocks, merged back when both are free
- Keep an array $A[0..K]$ where 2^K is the largest block size that can be allocated, each element is a linked list keeping track of free blocks of the appropriate size, indicated by starting address
- In actual implementation, there may be a smallest block size as well which is not cost effective to manage
- To allocate, find smallest 2^S that fits, then access it to find a free block
- If it exists, remove it from free block list and allocate
- If not, search from $S+1$ until a free block that that can fit is found, splitting it until we have a free block with size 2^S
- To free a block, check the appropriate array entry to see if buddy is free
- If buddy is free, remove both from list and merge to get a larger block, recursing upwards
- If buddy is not free, insert self to free block list
- Buddy of size 2^S has Sth bit as complement, with leading bits being the same, use XOR to find

Disjoint Memory Schemes

Paging Scheme

- Memory space can now be in disjoint physical memory locations, using a paging mechanism
- Physical Memory is split into regions of fixed size known as physical frames
- Logical memory of a process is similarly split into regions of same size known as logical page
- At execution, pages of a process are loaded into any available memory frame, while logical memory space remains contiguous
- Need lookup page table for mapping between logical page and physical frame
- Need to locate value in physical memory using logical memory address
- Physical Address= Physical Frame Number x Size of Physical Frame + Offset
- Keep frame size as power of 2, and physical frame size same as logical page size for simplicity
- Given page / frame size of 2^n and m bits of logical address
- $p = m - n$ bits of logical address, $d = n$ remaining bits of logical address
- Use p to find frame number f, to get physical address $PA = f \cdot 2^n + d$
- Paging removes external fragmentation since there is no left-over physical memory region or wastage
- But can still have internal fragmentation if logical memory space is not a multiple of page size
- Provides clear separation of logical and physical address space, allowing greater flexibility and simple address translation
- Software Implementation: OS stores page table information along with process information, but requires two memory accesses for every memory reference for frame number and actual access
- Hardware Implementation: Specialised on-chip component to support paging, known as Translation Look-Aside Buffer (TLB), acting as cache of a few page table entries
- Use page number to search TLB associatively, can have hit where frame number is retrieved to generate physical address or miss where memory access is needed to access full page table and update
- TLB is part of hardware context of a process, which gets flushed when context switching occurs
- When process resumes running, many TLB misses happen when repopulating
- Paging scheme can be extended to provide memory protection using access-right bits and valid bits
- Access Right bits attach to page table entries and indicate whether it is writable readable or executable, access is checked against this
- Not all processes use entire memory range, so valid bit is used to indicate whether it is valid for current process, attached to each page table entry
- Page Sharing: Page table can allow several processes to share the same physical memory frame, for code sharing or copy-on-write
- Copy-on-write: Locate free frame, clone original frame, update PTE to point to new frame and update W bit before performing write

Segmentation Scheme

- Memory space actually contains of multiple memory regions with different usage e.g. User Code region, Global Variables region, ...
- Some regions might grow and shrink at execution time, hard to place in contiguous memory space and check if memory access is valid
- Separate regions into multiple memory segments, each with name (usually a single number) and limit
- Memory references are now specified as Segment Name (Segment ID) + Offset
- Logical Address Translation: Use SegId to lookup Base, Limit in segment table, add with Offset to get Physical Address, use Limit to check validity
- Each segment is an independent contiguous memory space which can easily change size and be protected or shared
- Can cause external fragmentation

Segmentation with Paging

- Combine segmentation with Paging
- Each segment is now composed of several pages instead of a contiguous memory region
- Segment can grow by allocating a new page and adding to its own page table

Virtual Memory Management

- Split logical address space into pages, with some residing in physical memory, and others stored on secondary storage
- Use page table to translate between virtual and physical address
- Distinguish between memory resident (pages in physical memory) and non-memory resident (pages in secondary storage) using bit in page table entry
- CPU can only access memory resident pages, page fault occurs if it tries to access non-memory resident page, OS needs to bring it into physical memory
- Thrashing: Memory access results in page fault most of the time
- Temporal Locality: Memory address used recently is likely to be used again
- Spatial Locality: Memory addresses close to currently used one are likely to be used next
- Loading of pages should exploit this
- Separates logical memory address from physical memory
- Allows more efficient use of physical memory and more processes to reside in memory, improving CPU utilisation

Page Table Structure

- How to efficiently store large page tables
- Large page table causes high overhead and fragmentation
- Direct Paging: Keep all entries in a single table, with entries keeping track of physical frame number and additional information bits
- 2^p pages, use p bits to specify unique page
- 2-Level Paging: Split full page tables into regions, need directory to keep track
- Each page table is split into smaller page tables with page table number
- Single page directory is needed to locate smaller page tables
- If original has 2^P entries, each of the smaller 2^M page tables has $2^{(P-M)}$ entries
- Can have empty entries in page directory, reducing overhead
- Inverted Page Table: Keep a single mapping of physical frame to pid and page number
- By using one table for all processes, frame management is easier and faster, but translation is slow as whole table needs to be searched

Page Replacement Algorithms

- Policy to replace memory pages
- Clean: Not modified, Dirty: Modified, need to write back
- Memory references are often modeled as memory reference strings or sequences of page numbers

- Memory Access Time: $T_{access} = (1 - p) \cdot T_{mem} + p \cdot T_{pagefault}$
- p is probability of page fault, aim to reduce as much as possible
- Optimal Page Replacement: Replace page that will not be used again for the longest period of time, but needs future knowledge which is unrealistic
- FIFO: Evict oldest memory page based on loading time, easy to implement, but does not exploit temporal locality
- Belady's Anomaly: As physical frames increases, number of page faults somehow also increases, e.g. FIFO
- Least Recently Used: Uses temporal locality, expects page to be reused in short time window
- Not easy to implement as need to keep track of last access time
- Counter: Use logical time counter, incremented for every memory reference, have time-of-use field for page table entry
- Need to search through all pages, and time-of-use could overflow
- Stack: Maintain stack of page numbers, put on top if referenced, replace page at bottom of stack
- Not a pure stack, and hard to implement
- Second-Chance Page Replacement (CLOCK): Modified FIFO to give a second chance to pages that are accessed
- Each PTE now has reference bit
- Oldest FIFO page is selected, replaced if reference bit is 0
- If reference bit is 1, it is given a second chance and next FIFO page is selected
- Use circular queue to maintain the pages with a pointer pointing to oldest / victim page

Frame Allocation

- How to distribute limited frames among multiple processes
- Equal Allocation: Each process gets the same number of frames
- Proportional Allocation: Processes get a number dependent on memory usage
- Local replacement: Victim page selected among pages of process that causes page fault
- Frames allocated to a process is constant so performance is stable, but if not enough frames are allocated, progress is hindered
- Single process could hog I/O and degrade other process performance
- Global replacement: Victim page can be chosen among all physical frames, even from other processes
- Allows self adjustment between processes, but badly behaving processes can affect others, and can vary from run to run
- Cascading thrashing when processes start stealing pages from each other
- Working Set Model: Use locality observation where set of pages referenced by a process is relatively constant in a period of time
- Define an interval of time known as working set window Δ
- $W(t, \Delta) =$ active pages in interval of time t
- Allocate enough pages for those in $W(t, \Delta)$ to reduce possibility of page fault
- Transient Region: Working set changing in size
- Stable Region: Working set about the same for a long time
- Accuracy depends on choice of Δ, may miss pages or have extra pages

File System

- Use external storage to store persistent information
- Direct access to storage media is not portable, depends on hardware
- File system provides abstraction on top of physical media, high level resource management scheme, protection and sharing between processes and users
- General Criteria: Self-Contained (Information stored on media should be enough to describe entire organisation), Persistent (Beyond the lifetime of OS and processes), Efficient (Provides good management of free and used space, minimum overhead of bookkeeping information)

File System Abstraction

- File System: Consists of a collection of files and directory structures
- File: Logical unit of information created by process, contains data and metadata / file attributes
- File Type: Dependent on OS, each has an associated set of operations such as specific program for processing e.g. regular files, directories, special files
- ASCII Files: Can be displayed or printed as is
- Binary Files: Have predefined internal structure that can be processed by specific program
- Distinguish using file extension or embedded information such as magic number
- File Protection: Controlled access to information stored in a file, restrict access based on user identity
- Types of access: WRX, append, delete, list metadata
- Access Control List: List of user identity and allowed access types, very customisable but additional information associated with each file
- Permission Bits: Classify users as owners, group, or universe, define read/write/executable permission for each of these
- File metadata can be changed, read, or file can be renamed
- File Structure: Array of bytes with offset, can be fixed length record (easy to locate), or variable length records
- Sequential Access: Data read in order, starting from beginning, cannot skip but can be rewound
- Random Access: Data can be read in any order; read or seek of offset
- Direct Access: Used for file with fixed length records, allows random access to records, useful when there are large amount of records
- OS provides file operations as system calls, providing protection, concurrent and efficient access
- For open file, file pointer, disk location, and open count is kept
- 2 table approach: System-wide open-file table (Keep track of open files in system), Per-process open-file table (Keep track of open files for a process, each entry points to the system-wide table entries)
- File descriptors could possibly be shared

Directory

- Use to provide logical grouping and keep track of files
- Different ways to structure: Single-Level, Tree-Structure, DAG, General Graph
- Tree-Structure: Directories can be recursively embedded in other directories, naturally forming a tree structure
- File can be referred to using absolute or relative pathname from root or current working directory
- DAG: File can be shared and appear in multiple directories although only one copy actually exists through aliases
- Hard Link: Limited to file only, different directories have different pointers to actual file in disk
- Low overhead, only pointers added, but causes deletion problems
- Symbolic Link: Special link file used, when accessed, locates and accesses original file, contains path name
- Deletion is simple, but larger overhead
- General Graph: Hard to traverse and delete
- R: Can you read this list, W: Can you change this list, X: Can you use this directory as PWD (list is list of directory entries)

File System Implementation

- File system stored on storage media
- General Disk Structure: 1-D array of logical blocks (smallest accessible unit), each mapped into disk sectors (layout hardware dependent)
- Master Boot Record (MBR) at sector 0 with partition table, followed by one or more partitions which can contain an independent file system
- File system contains OS Boot-Up information, Partition details (Blocks and Locations), Directory Structure, File information, Actual file data in a partition after MBR

File Implementation

- File is a collection of logical blocks, can cause internal fragmentation
- Keep track of logical blocks to allow efficient access and utilise disk effectively
- Contiguous Allocation: Allocate consecutive disk blocks to a file
- Simple to keep track, each file only needs starting block number and length, fast access, but can cause external fragmentation
- Linked List: Keep a linked list of disk blocks, each storing next disk block number and actual file data
- File information stores first and last disk block number
- Solves fragmentation problem, but random access in a file is very slow, and part of disk block is used for pointer, less reliable
- File Allocation Table (FAT): Move all block pointers into a single table, simple and efficient, providing fast random access
- Keeps track of all disk blocks in a partition, can be large in size and consume extra memory space
- Indexed Allocation: Files have index block, an array of disk block addresses
- Lesser memory overhead, only index block of opened file needs to be in memory, provides fast direct access
- Limited maximum file size, and has index block overhead
- Linked Scheme: Keep linked list of index blocks, each containing pointer
- Multilevel Index: Similar to multi-level paging, first level index block points to a number of second level index blocks, each pointing to an actual disk block
  Combined: Combination of direct indexing and multi-level index scheme

Free Space Management

- Maintain free space list to perform file allocation
- Bitmap: Each disk block is represented by a single bit for its status
- Provides a good set of manipulations, but need to keep in memory for efficiency
- Linked List: Used a linked list of disk blocks containing a number of free disk block numbers and pointer to next free space disk block
- Easy to locate free block, and only first pointer is needed in memory, but has high overhead

Implementing Directory

- Directory keeps track of files in directory, mapping file name to file information
- Given full path name, recursively search directories to arrive at file information
- Sub-directory usually stored as file entry with special type in a directory
- Linear List: Directory consists of a list with each entry storing a file name and other metadata as well as file information or pointer to it
- Locating a file requires a linear search, inefficient for large directories, latest few searches can be cached
- Hash Table: Directory is hash table, with file name hashed into it, using chaining
- Fast lookup, but hash table has limited size and depends on good hash function
- File Information: File name and metadata, disk blocks information
- Store everything in directory entry, with fixed size entries
- Can store only file name and point to another data structure for other information

File System in Action

- When user interacts with file, run-time information is needed which is maintained by OS in memory
- Create File: Use full pathname to locate parent directory, search for filename to avoid duplicates, cache search on directory structure
- Use free space list to find free disk blocks, then add an entry to parent directory
- Open File: Search system wide table for existing entry; if found, create an entry in the open file table to point to it, if not, use full pathname to locate it
- If located, load file information into new entry in system wide table, then create an entry in the process table
- Use returned pointer for further reads and writes